AI4201: Machine Learning in Engineering

Programming Assignment

Perceptron and ADALINE from Scratch

Student ID:

Course: AI4201

Models: Perceptron and ADALINE

Random seed: 42

Assignment overview

A binary Iris classification study comparing two NumPy implementations, their convergence, decision boundaries, test performance and robustness to simulated sensor noise. Includes the optional three-class One-vs-Rest extension.

1. Dataset acquisition and audit

Dataset and engineering context

Iris measurements are treated as plant-inspection sensor readings. The binary experiment retains Setosa and Versicolor, maps their labels to -1 and +1, and uses sepal length and petal length.

Task 1: Dataset Documentation and Data Quality

Dataset Source: UCI Machine Learning Repository – Iris Dataset.

https://archive.ics.uci.edu/dataset/53/iris

Feature Units: All four flower measurements are recorded in centimeters (cm).

Data Quality Risk: Real-world sensors may produce inaccurate measurements due to calibration errors or sensor noise. These errors could affect the classification accuracy of the models.

Source note: The notebook first attempts the official UCI CSV download. When network access is unavailable, it transparently uses the packaged scikit-learn copy and reports the fallback. The final experiment should be rerun with internet access to demonstrate the official download.

2. Binary exploration and preprocessing

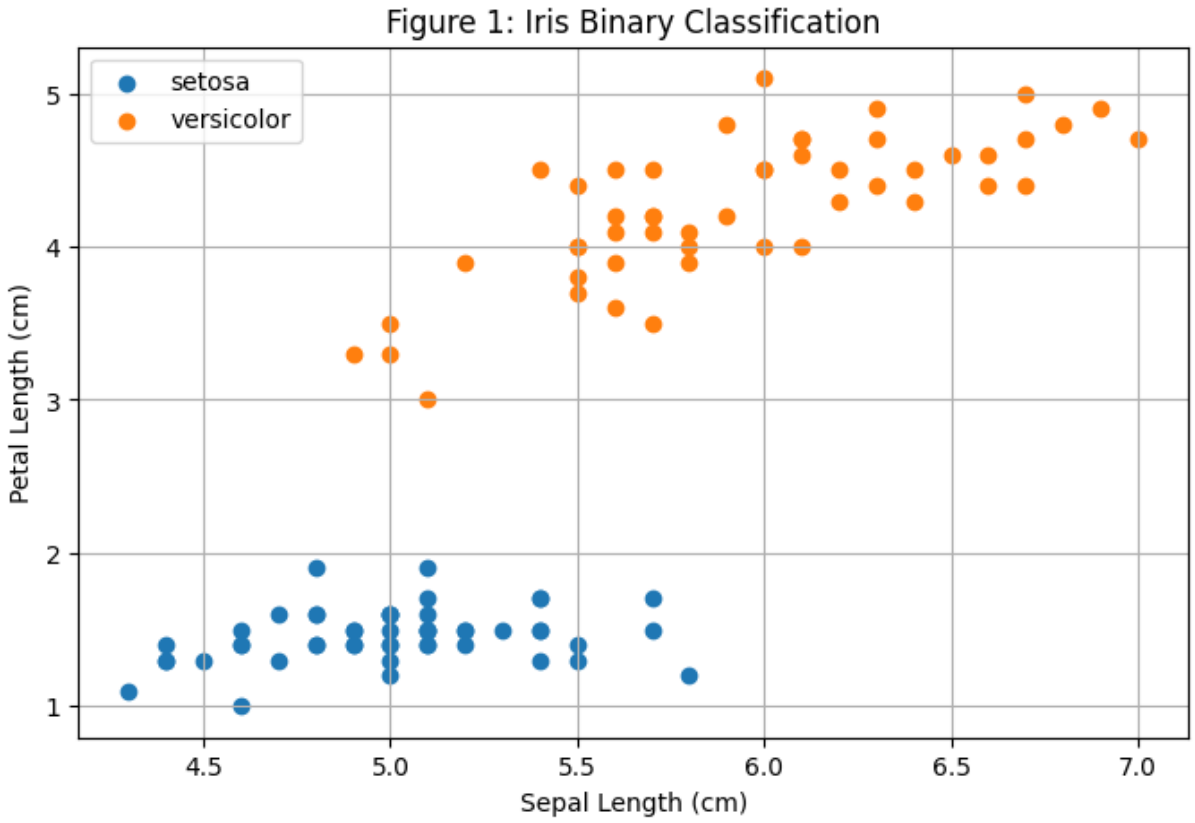


Figure 1. Iris Setosa and Versicolor by sepal and petal length (cm).

Figure 1 interpretation: The two classes are separated mainly along petal length; the plot displays centimeters on both axes.

Task 2: Feature Scaling Discussion

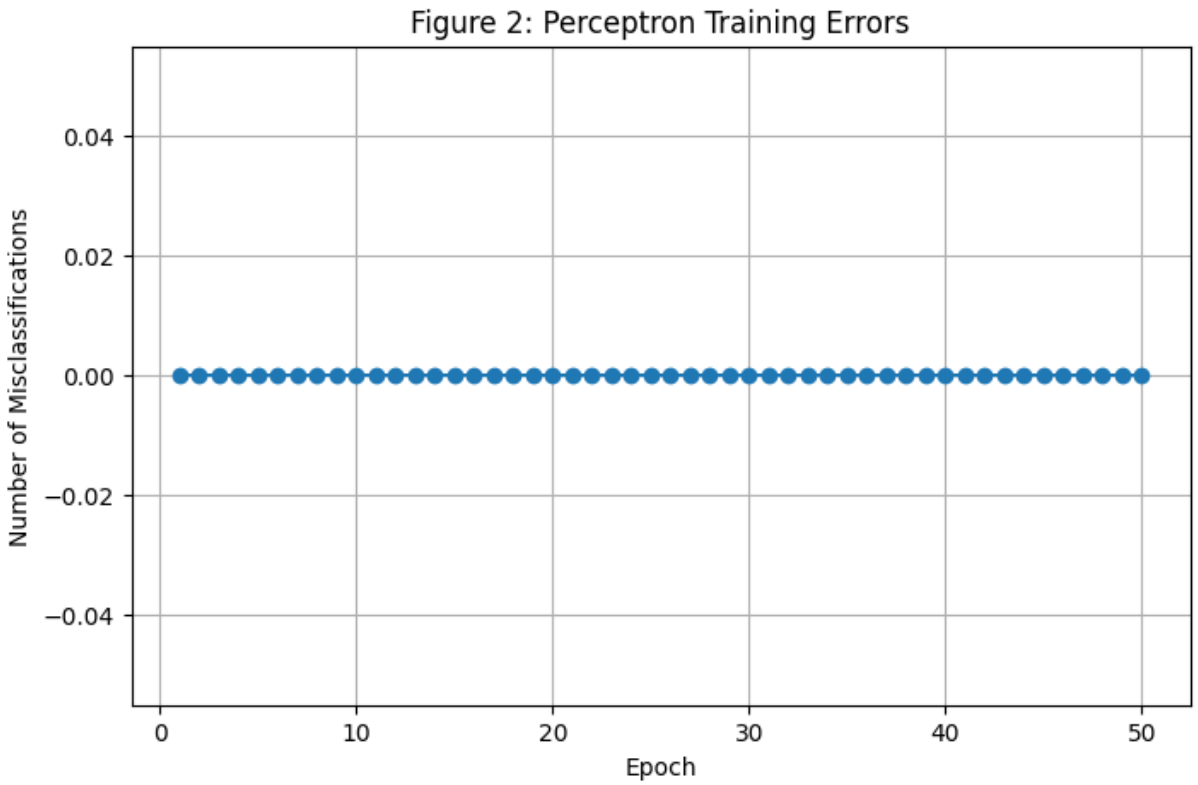
ADALINE is generally more sensitive to feature scaling because it uses gradient descent to minimize squared error. Features with different scales can cause unstable or slow convergence. Standardization helps improve training stability.

The Perceptron can also be affected by feature scaling, especially in its convergence speed and update behavior.

3. Perceptron from scratch

Perceptron learning rule

Prediction uses the sign of z = wT x + b. For each sample, update w by eta (y - yhat) x and b by eta (y - yhat).



Perceptron training errors (initial experiment).

Task 3: Manual Perceptron Update

Given:

• Input: x = [1, 2]

• True label: y = +1

• Initial weights: w = [0, 0]

• Initial bias: b = -1

• Learning rate: η = 0.01

Step 1: Calculate net input

z = (0 × 1) + (0 × 2) – 1 = –1

Since z < 0, the predicted label is –1.

Step 2: Calculate update

η(y – ■) = 0.01 × (1 – (–1)) = 0.02

Step 3: Update weights and bias

w_new = [0, 0] + 0.02 × [1, 2] = [0.02, 0.04]

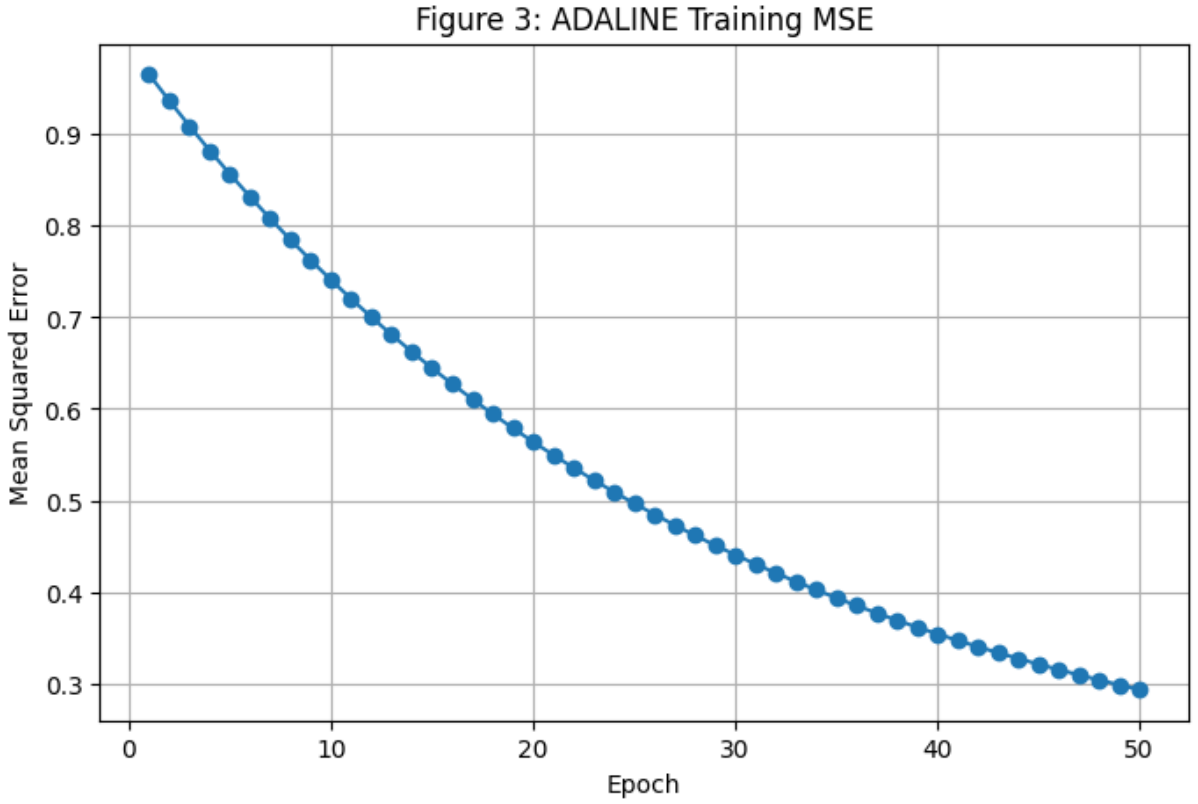
b_new = –1 + 0.02 = –0.98

The manual calculation matches the NumPy implementation.

4. ADALINE batch gradient descent

ADALINE learning rule

Batch gradient descent minimizes J = (1 / 2n) sum (y - z)^2 using the continuous linear output z. The threshold is used only for class predictions.



ADALINE mean squared error (initial experiment).

Task 4: ADALINE Manual Batch Update

The ADALINE model uses batch gradient descent to minimize squared error.

For the manual example:

• X = [[1, 2], [2, 1]]

• y = [1, -1]

• Initial weights = [0, 0]

• Initial bias = 0

• Learning rate = 0.1

Step 1: Calculate linear outputs

z = [0, 0]

Step 2: Calculate errors

y – z = [1, –1]

Step 3: Update weights

X■(y – z) = [–1, 1]

w_new = [0, 0] + (0.1 / 2) × [–1, 1]

w_new = [–0.05, 0.05]

Step 4: Update bias

b_new = 0 + 0.1 × (1 – 1) / 2 = 0

The manual batch update matches the NumPy gradient calculation. The implementation is also tested using a one-epoch fit with its seeded initial weights.

Unlike Perceptron, ADALINE uses the continuous linear output during training and applies the classification threshold only during prediction.

5. Controlled training experiments

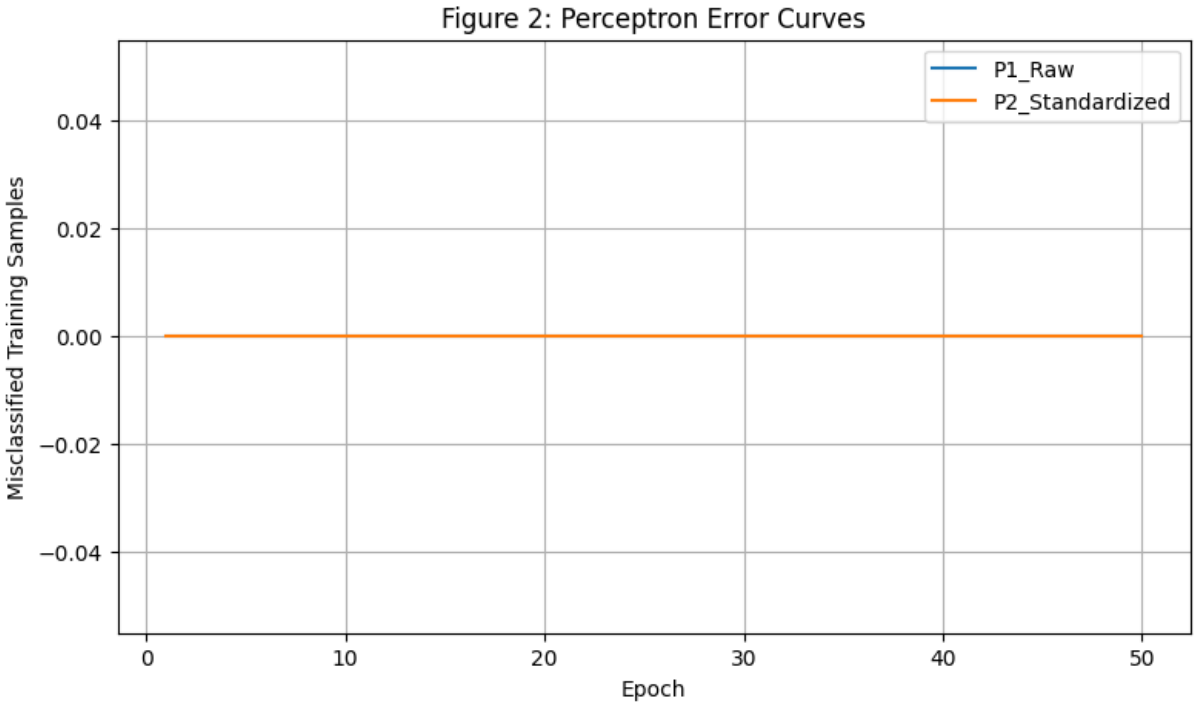


Figure 2. Perceptron error curves over 50 epochs.

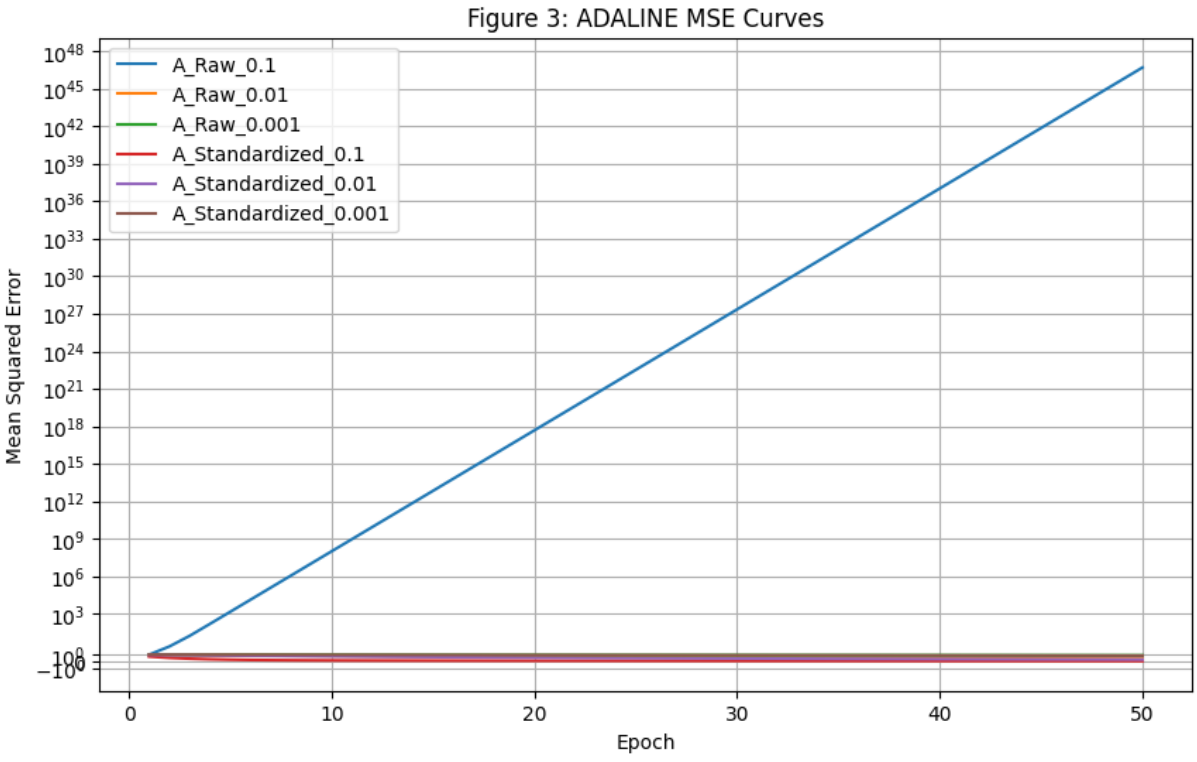


Figure 3. ADALINE mean squared error curves.

Figure 2 interpretation: Compare the number of misclassified training examples over 50 epochs for raw and standardized Perceptron runs.

Figure 3 interpretation: Compare ADALINE squared-error convergence across three learning rates and both feature scales; rising cost suggests instability.

Model Selection Criteria

The final models will be selected using training results only, without inspecting test-set performance.

For Perceptron, the configuration with the fewest misclassified training samples after 50 epochs will be selected.

For ADALINE, the configuration with the lowest finite final training MSE after 50 epochs will be selected. Divergent runs will be excluded.

If configurations tie, the standardized configuration will be preferred, followed by the smaller learning rate where applicable.

The test set will be reserved for the final evaluation.

Task 5: Experiment Interpretation

The experiments compare the effects of feature scaling and learning rate on Perceptron and ADALINE.

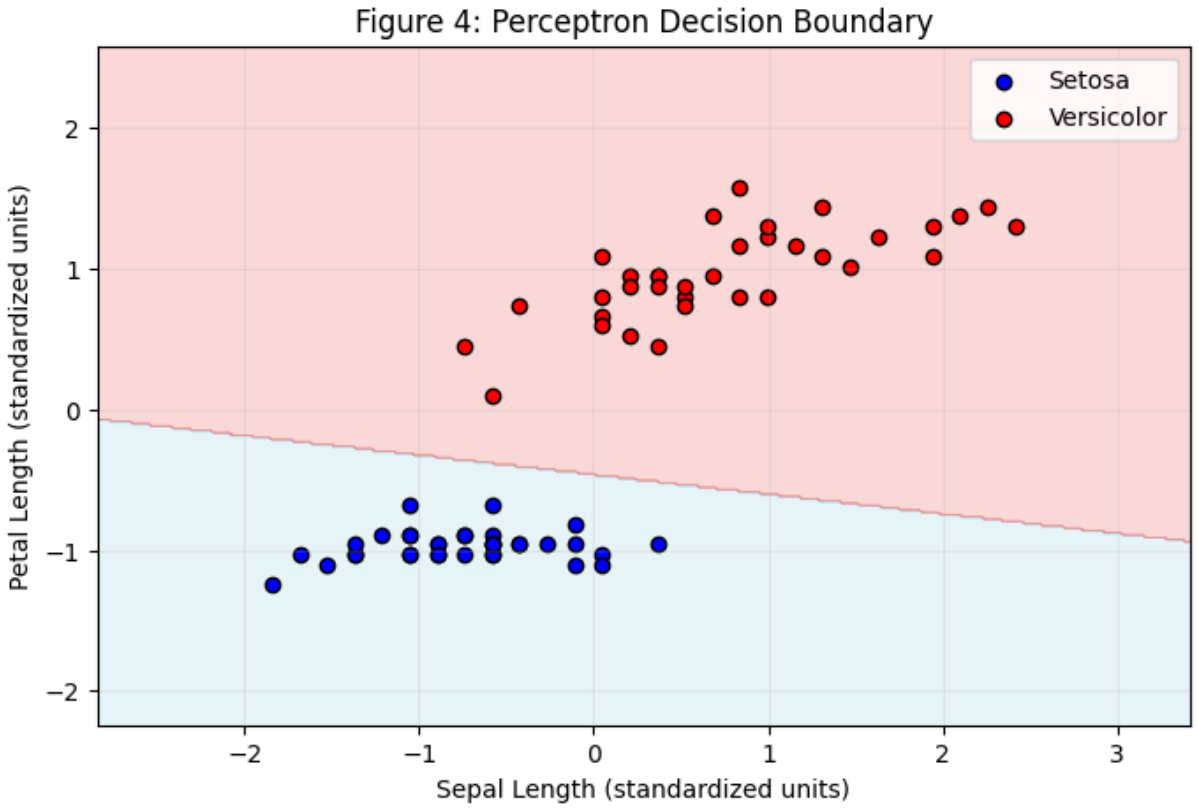
Perceptron is evaluated using the number of misclassified training samples after each epoch, while ADALINE is evaluated using mean squared error.

The final configurations are selected according to the predefined training criteria. The test data has not been used for model selection.

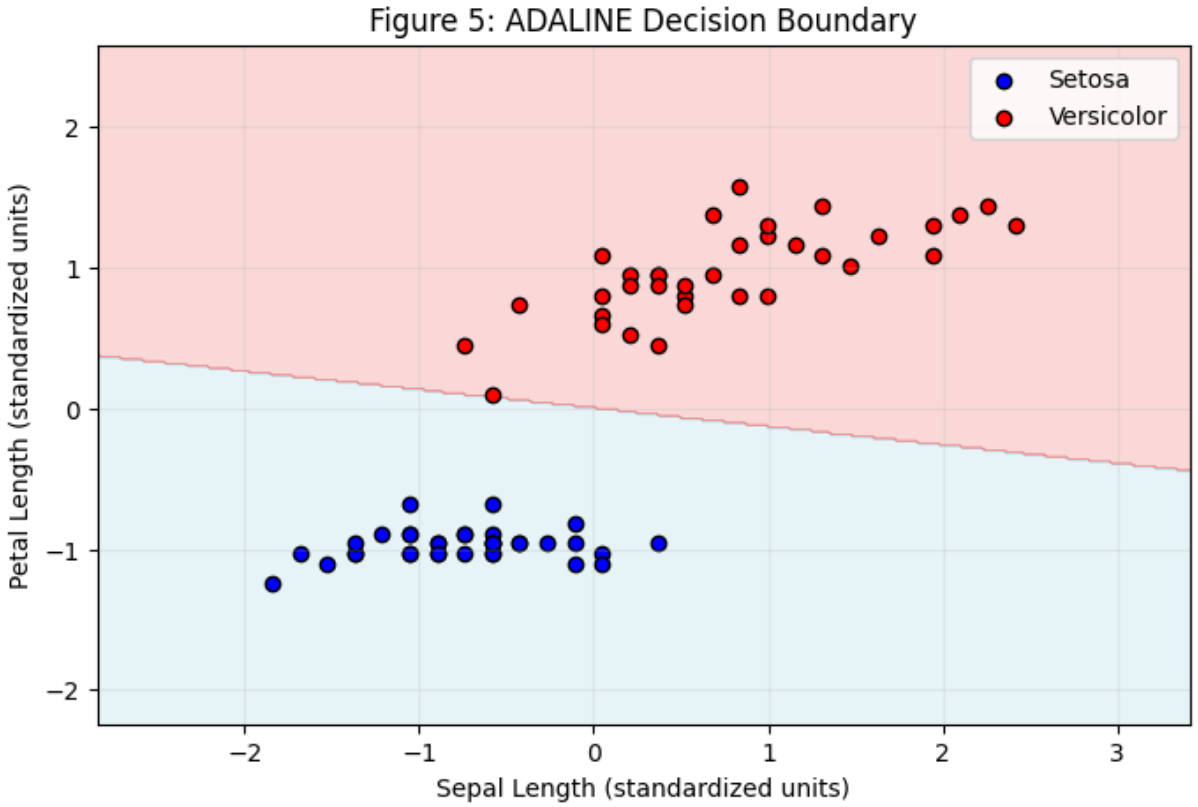
Selected configurations: P2_Standardized for Perceptron and A_Standardized_0.1 for ADALINE.

6. Visual evaluation and comparison

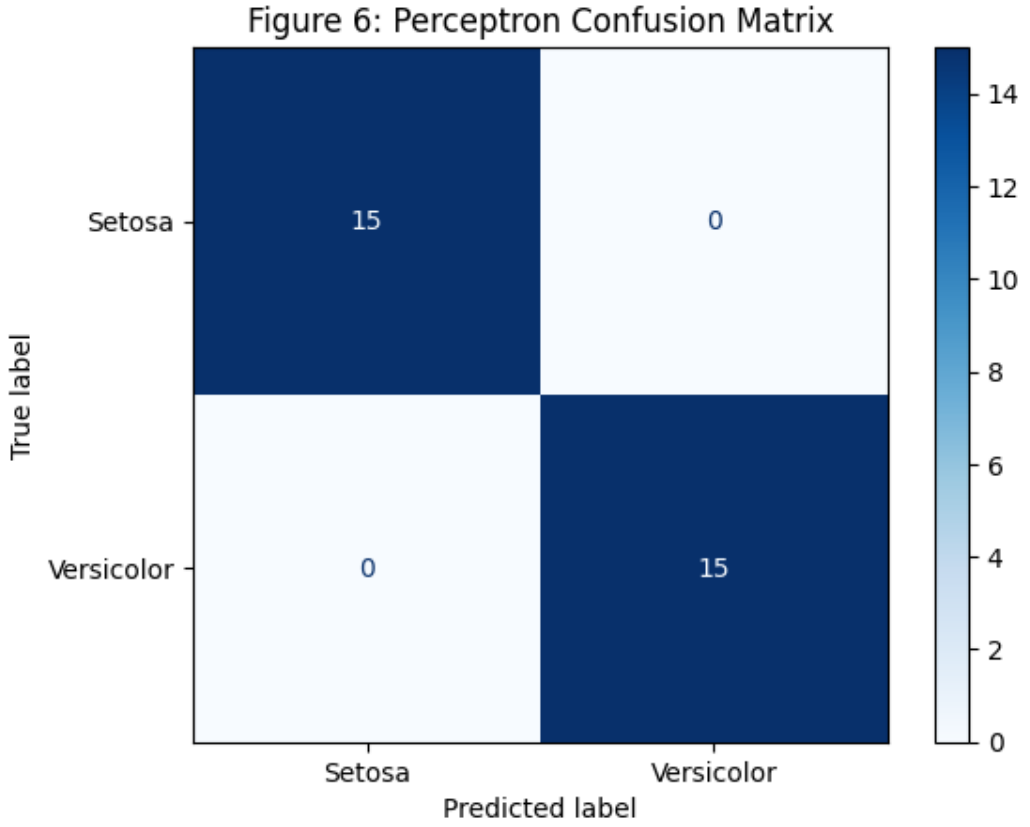
Model	Accuracy	Precision	Recall	F1
Perceptron	1.000	1.000	1.000	1.000
ADALINE	1.000	1.000	1.000	1.000



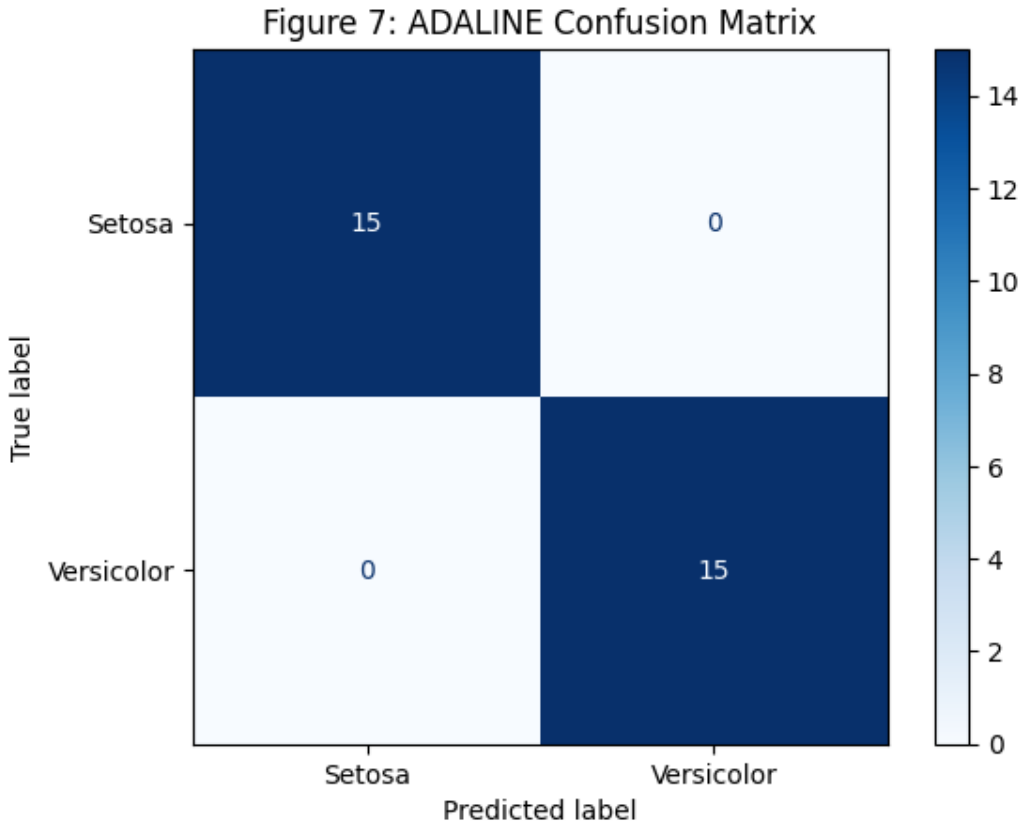
Figures 4–5. Learned decision boundaries for both models.



Figures 4–5. Learned decision boundaries for both models.



Figures 6–7. Test confusion matrices.



Figures 6–7. Test confusion matrices.

Figures 4–5 interpretation: Both models learn linear boundaries, but their different optimization rules can place the boundaries differently.

Figures 6–7 interpretation: The confusion matrices show the number of correctly and incorrectly classified Setosa and Versicolor test samples.

Task 6: Model Evaluation and Comparison

The selected Perceptron and ADALINE models were evaluated using the held-out test dataset.

Accuracy, precision, recall, and F1-score were calculated to assess classification performance. Decision-boundary plots and confusion matrices were also generated.

Both models produce linear decision boundaries, but their training objectives are different. Perceptron updates its parameters based on classification errors, whereas ADALINE minimizes squared error using continuous linear outputs.

Therefore, the two models can learn different weights, decision boundaries, and convergence curves even if they achieve identical test accuracy.

Task 6: Evaluation Results and Interpretation

Both Perceptron and ADALINE achieved 100% accuracy, precision, recall, and F1-score on the test dataset.

Both models correctly classified all 30 test samples, with no misclassifications.

Although their classification performance was identical, the models use different learning rules. Perceptron updates its parameters based on classification errors, while ADALINE minimizes squared error using batch gradient descent.

Perfect accuracy on this small test set does not guarantee perfect performance on unseen or noisy sensor measurements.

7. Simulated sensor-noise study

Model	Noise σ	Mean accuracy	Std. deviation
ADALINE	0.10	1.000000	0.000000
ADALINE	0.25	0.982222	0.020960
ADALINE	0.50	0.972222	0.032851
Perceptron	0.10	1.000000	0.000000
Perceptron	0.25	0.992222	0.014339
Perceptron	0.50	0.955556	0.037481

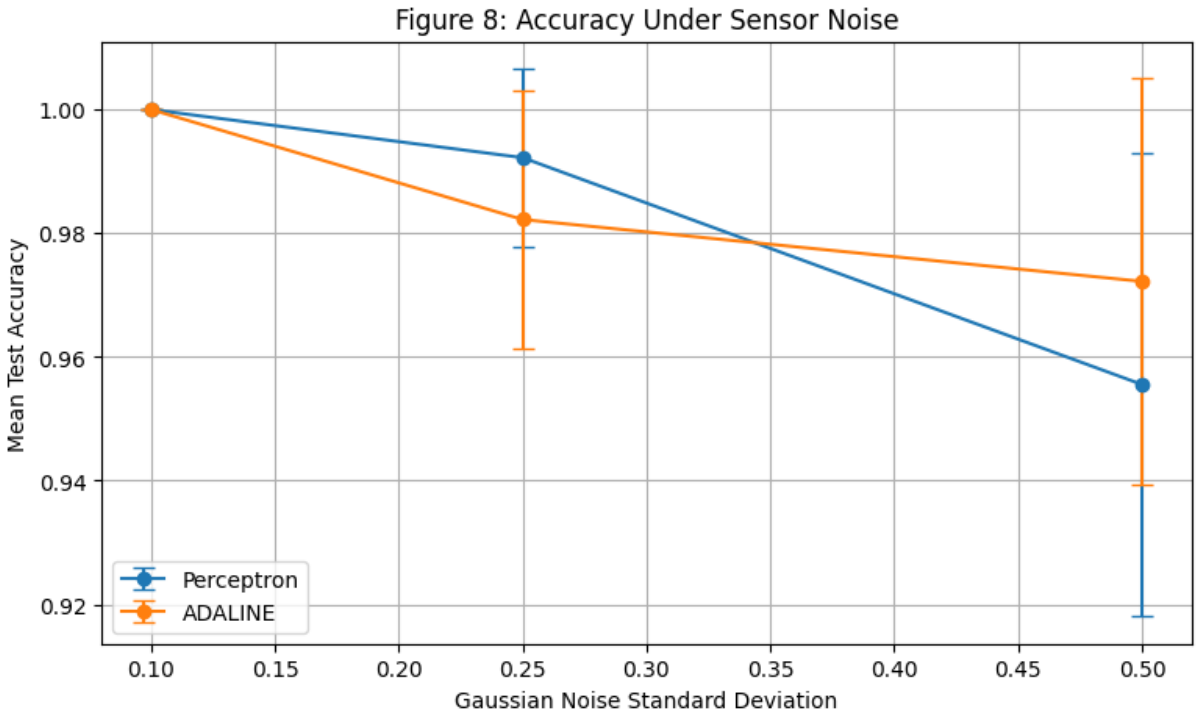


Figure 8. Mean accuracy under Gaussian noise, with error bars.

Figure 8 interpretation: Accuracy decreases with stronger perturbations; at σ=0.50, ADALINE had a higher mean accuracy in this experiment.

Task 7: Sensor Noise Experiment

Gaussian noise with zero mean was added to the standardized test features at three standard deviations: 0.10, 0.25, and 0.50.

Each noise level was tested 30 times without retraining either model.

The mean accuracy and standard deviation were calculated to evaluate the robustness of Perceptron and ADALINE under simulated sensor measurement errors.

The models were trained using different objectives: Perceptron minimizes classification mistakes through its update rule, whereas ADALINE minimizes squared error. These differences may influence how their decision boundaries respond to noisy inputs.

Task 7: Noise Robustness Results and Discussion

Both Perceptron and ADALINE achieved 100% mean accuracy at a noise standard deviation of 0.10.

At σ = 0.25, Perceptron achieved 99.22% mean accuracy, compared with 98.22% for ADALINE.

At σ = 0.50, ADALINE achieved 97.22% mean accuracy, outperforming Perceptron, which achieved 95.56%.

These results suggest that ADALINE was more robust at the highest tested noise level, while Perceptron performed slightly better at the intermediate level.

ADALINE minimizes squared error using continuous outputs, whereas Perceptron updates its weights based on classification errors. These learning objectives can produce different decision boundaries and sensitivities to noise.

However, the differences are relatively small, and this experiment alone is not sufficient to establish that either model is universally more robust.

8. Engineering interpretation and reproducibility

1. Model Selection for Deployment

For this binary plant-inspection problem, I would initially select ADALINE as the preferred model for deployment. Both Perceptron and ADALINE achieved 100% accuracy, precision, recall, and F1-score on the clean test dataset. However, their performance under simulated sensor noise was slightly different.

At the highest tested noise level ($\sigma = 0.50$), ADALINE achieved 97.22% mean accuracy, while Perceptron achieved 95.56%. This suggests that ADALINE may offer better robustness under stronger measurement noise in this particular experiment. Nevertheless, additional testing would be necessary before deployment.

2. Limitations of Accuracy and Dataset Size

Accuracy alone is not sufficient to evaluate a machine-learning model. A model may achieve high accuracy while still making important mistakes on particular classes or difficult measurements. Therefore, precision, recall, F1-score, and confusion matrices should also be considered.

The Iris dataset is relatively small, and this binary experiment uses only 100 samples from two flower species. The test set contains 30 samples, which limits confidence in how well the results generalize to real industrial environments. Real sensor measurements may contain more noise, missing values, unexpected conditions, and different distributions.

3. Data Drift and Sensor Calibration

Data drift occurs when the statistical properties of incoming measurements change over time. For example, changes in environmental conditions, equipment aging, or plant characteristics could affect the measurements collected by sensors.

Sensor calibration errors could introduce systematic measurement shifts. Such shifts may move observations across the learned decision boundary and increase classification errors. Therefore, regular calibration and data-quality checks would be necessary.

4. Monitoring and Retraining

Two important monitoring signals would be classification performance and changes in input-feature distributions.

First, accuracy or recall should be monitored using periodically verified inspection samples. A sustained decrease could indicate declining model reliability.

Second, the distributions of sepal length and petal length measurements should be compared with the original training data. Significant changes may indicate sensor problems or data drift.

A possible retraining trigger would be a sustained decrease in verified accuracy below 95%, after investigating measurement quality and confirming that the decline is not caused by a temporary sensor issue.

5. Ethical and Operational Risk

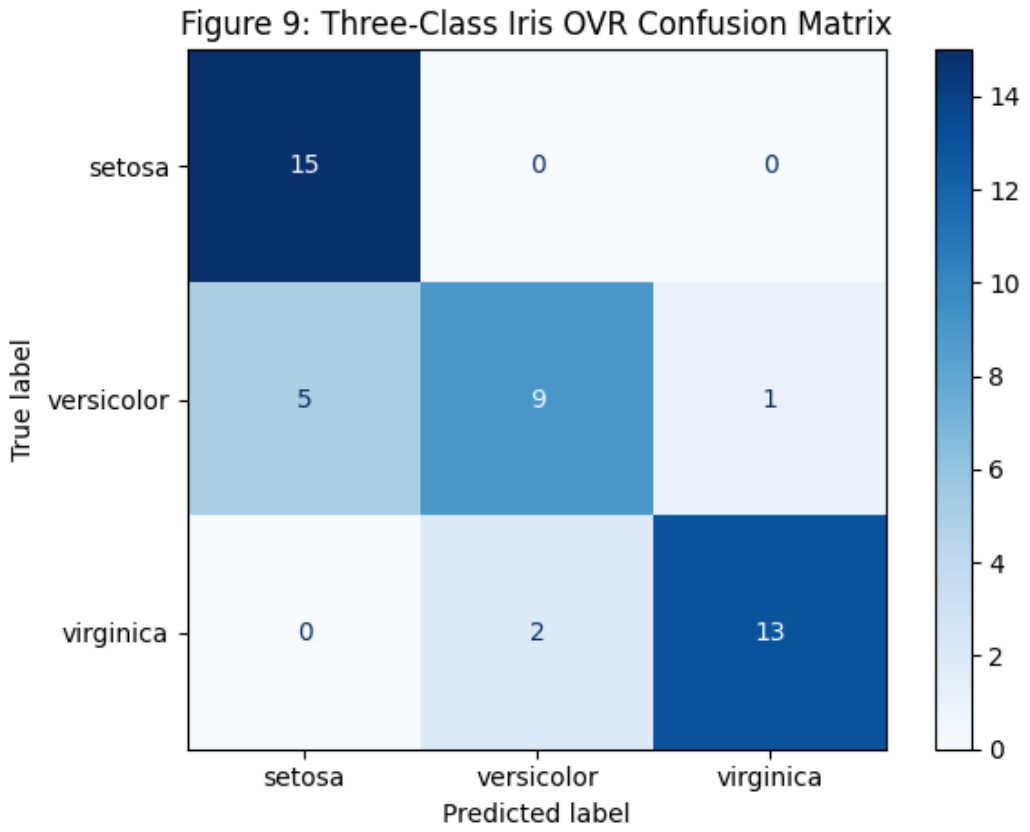
An important operational risk is making incorrect automated inspection decisions. Misclassifying a plant could result in unnecessary rejection or acceptance, potentially causing financial losses or reducing inspection reliability.

To reduce this risk, uncertain predictions should be reviewed by a human operator, particularly during the initial deployment period. The system should also maintain records of predictions, sensor measurements, and model versions for traceability.

Conclusion

Although both models performed perfectly on the clean test data, ADALINE showed slightly better robustness at the highest simulated noise level. I would therefore choose ADALINE as a provisional deployment candidate, while requiring further validation using larger datasets and real sensor measurements.

Optional bonus: three-class One-vs-Rest



Bonus. Three-class One-vs-Rest confusion matrix.

Optional Bonus (+10): Three-Class One-vs-Rest Extension

We extend our own PerceptronScratch to classify all three Iris species using all four measurements (cm). Each of three binary classifiers treats its target species as +1 and the remaining two as −1. The predicted class is the classifier with the largest linear net input (score). The 70/30 stratified split uses seed 42, and scaling is fitted on training data only. No ready-made classifier is used.

Bonus interpretation

The binary task distinguishes setosa from versicolor using two features and reached an F1-score of 1.00 on the held-out test set. The three-class extension adds virginica and uses all four features; its macro-F1 averages F1 across all three species, giving each class equal importance. The confusion matrix identifies which pairs were mistaken for one another on the three-class test set. If no off-diagonal errors appear, the test results cannot establish a hardest pair; it would be inappropriate to claim one from these results alone. This comparison is not controlled for number of classes or number of features, so differences cannot be attributed solely to the algorithm.

Observed results: On this seeded test split, the OVR model achieved macro-F1 = 0.8153 and accuracy = 0.8222 (37/45 correct). The largest observed mutual confusion was setosa vs versicolor, with 5 errors (5 versicolor samples predicted as setosa). The binary test achieved F1 = 1.00 on 30 samples. The three-class task is harder here, but it also uses four rather than two features, so these are not directly equivalent test conditions. The result is specific to this training configuration and split.

Bonus results: macro-F1 = 81.53%; accuracy = 82.22% on 45 test samples.